

BICOL UNIVERSITY
College of Science
Computer Science and Information Technology Department
Legazpi City

BUCS Messaging System

A Final Project
in
Web Systems and Technologies

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Information Technology

Besimo, Marley D.
Postrado, Mishli Valorrhi N.
Sulpico, Shieanne R.
Gebilaguin, Natalie A.
BSIT - 2B

2026

Table of Contents

Project Overview.....	3
CRUD Functionality.....	3
Technology Stack.....	4
Database Design.....	5
Module Features.....	5
Individual Reflection.....	9

WEB SYSTEMS AND TECHNOLOGIES

FINAL PROJECT/EXAM

Project Overview

The BUCS Messaging System is a web-based messaging platform recreated from the original system developed by Pasiona and Fajardo in 2010 for the Bicol University College of Science. The system was designed to improve communication by allowing users to send and receive messages, and share files within a centralized platform. As students, communication is important for academic activities such as assignments, group projects, and information sharing, making a messaging platform useful for providing organized and accessible communication among users. For the final project in Web Systems and Technologies, the main focus is the Message Management Module, specifically the implementation of CRUD (Create, Read, Update, Delete) functionalities. The module enables users to create or send messages, view existing messages, edit message information, and delete messages from the system.

1. CRUD Functionality

The project implements the four major CRUD operations:

- **Create.** Ability to add new records with form validation
- **Read.** Display records in a table or list with search/filter capabilities
- **Update.** Edit existing records with confirmation
- **Delete.** Remove records with safety measures (confirmation dialog)

Technical Documentation

1. Project Overview

Original Project Title: BUCS Messaging System

Original Authors/Source:

PASIONA, GAIL D. & FAJARDO, ERICKZON S. "BU (Unpublished Undergraduate Thesis, Bicol University College of Science, Legazpi City, March 2010)

Module Scope: The module focuses on the management of messages, including creating, viewing, updating, and deleting records stored in a database.

Rationale: The proponents selected this module because messaging systems are widely utilized in academic institutions for communication and information dissemination. Recreating this module provides practical experience in database management, CRUD implementation, user interface design, and backend processing, while enhancing technical skills in web-based system development.

2. Technology Stack

The BUCS Messaging System was developed using different web technologies to support both front-end and back-end operations. These technologies were selected to ensure efficient message management, database connectivity, and user interaction.

- **CSS.** Cascading Style Sheets (CSS) was utilized to define the visual presentation and layout of the system interface. It was used to enhance the appearance of webpages by specifying styles for colors, typography, spacing, responsiveness, and overall user interface consistency.
- **PHP.** Hypertext Preprocessor (PHP) served as the primary server-side scripting language of the system. It was responsible for processing user requests, implementing CRUD (Create, Read, Update, Delete) operations, handling session management, and establishing communication between the application and the database.
- **phpMyAdmin.** phpMyAdmin was utilized as a database administration tool for managing the system database. It provided a graphical user interface (GUI) for executing queries, importing database files, managing tables, and monitoring stored records.
- **XAMPP.** XAMPP functioned as the local development environment used during system implementation and testing. It provided an integrated package consisting of Apache Web Server, PHP interpreter, and MySQL/MariaDB database services required for deploying and running the application in a local host environment.
- **Apache Web Server.** Apache was used as the web server responsible for hosting and processing the system application. It enabled communication between the client-side interface and server-side scripts, allowing users to access and interact with the system through a web browser.
- **Visual Studio Code.** Visual Studio Code was utilized as the integrated development environment (IDE) for source code editing, debugging, and project management throughout the development process.

3. Database Design

The BUCS Messaging System uses a relational database to manage and store message-related data efficiently. The database is designed to support CRUD operations (Create, Read, Update, Delete) while ensuring data integrity and proper relationship handling between tables.

Entity Relationship Diagram:

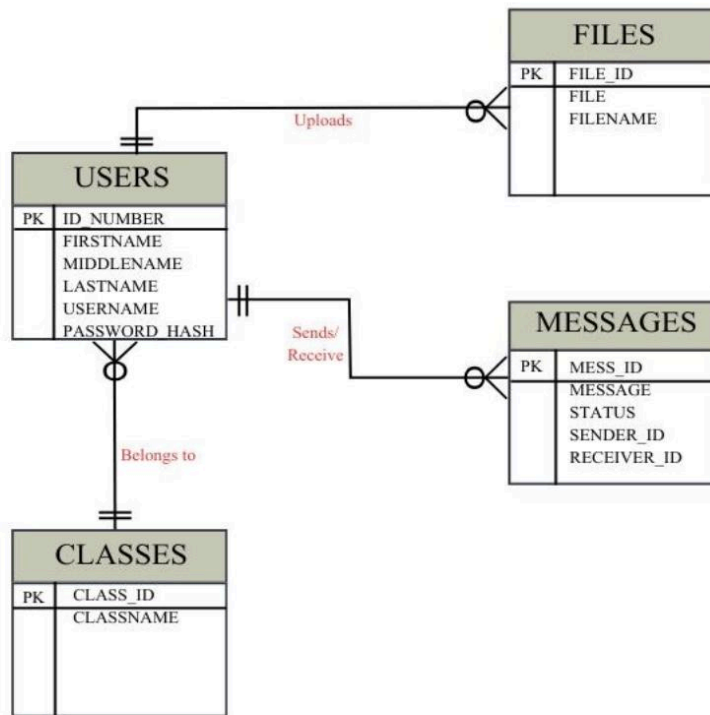


Figure 1. Entity-Relationship Diagram (ERD)

4. Module Features

The BUCS Messaging System implements the four fundamental CRUD (Create, Read,

Update, and Delete) operations to enable efficient message management. Each operation plays an essential role in ensuring that users can properly manage messaging records within the system.

- **CREATE.** Users are able to compose messages and upload files through the messaging system. This feature allows insertion of new message records and files into the database using an input form with validation mechanisms.

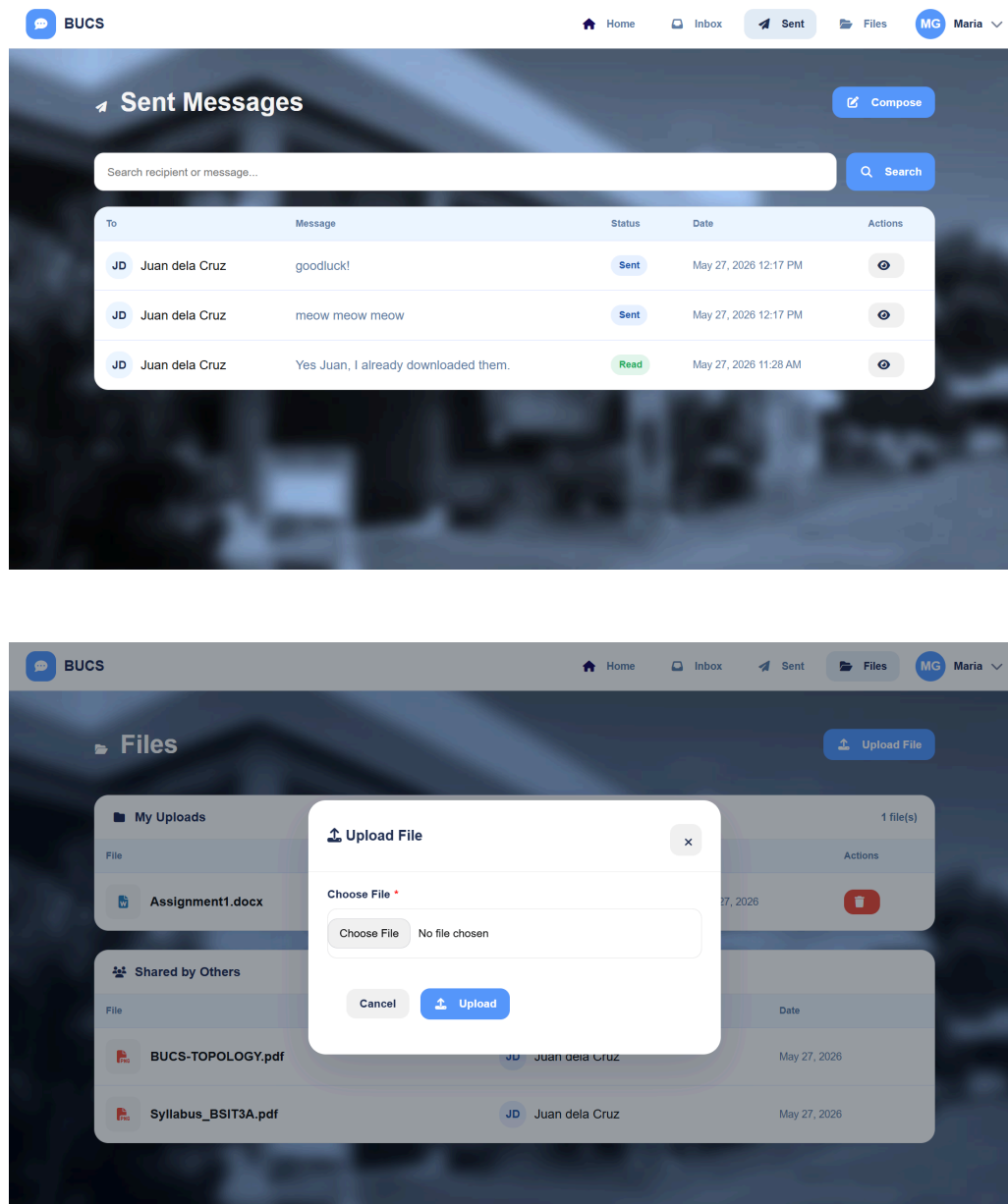


Figure 2. Create Function Interface

- **READ.** Users are able to view stored messages through the messaging system. It displays message records and retrieves information.

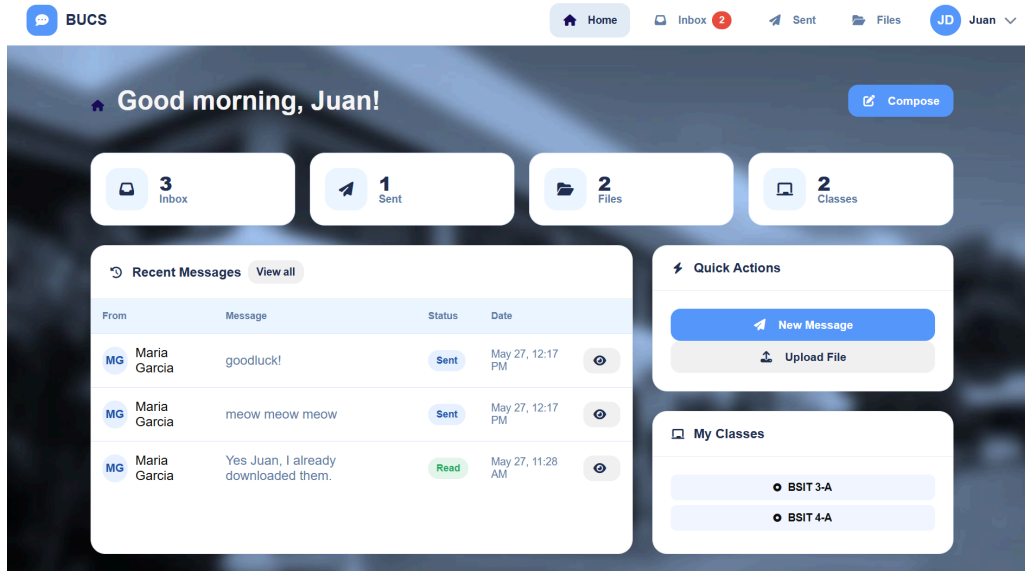


Figure 3. Read Function Interface

- **UPDATE.** Users are able to modify existing message records and allow updates to message information whenever necessary.

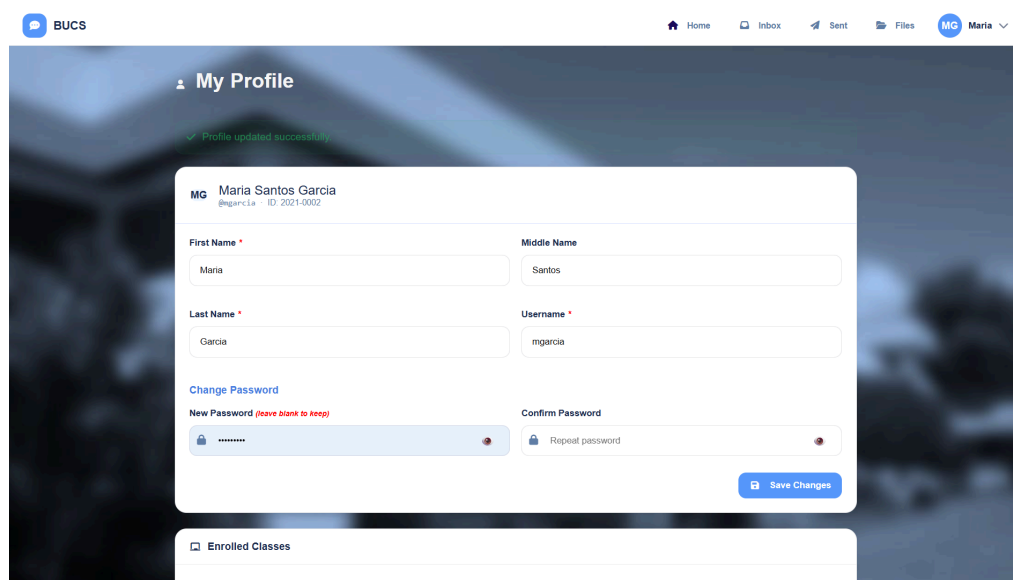


Figure 4. Update Function Interface

- **DELETE.** Users are able to remove selected messages through the messaging system. Safety measures are implemented to prevent unintended modification and deletion of records.

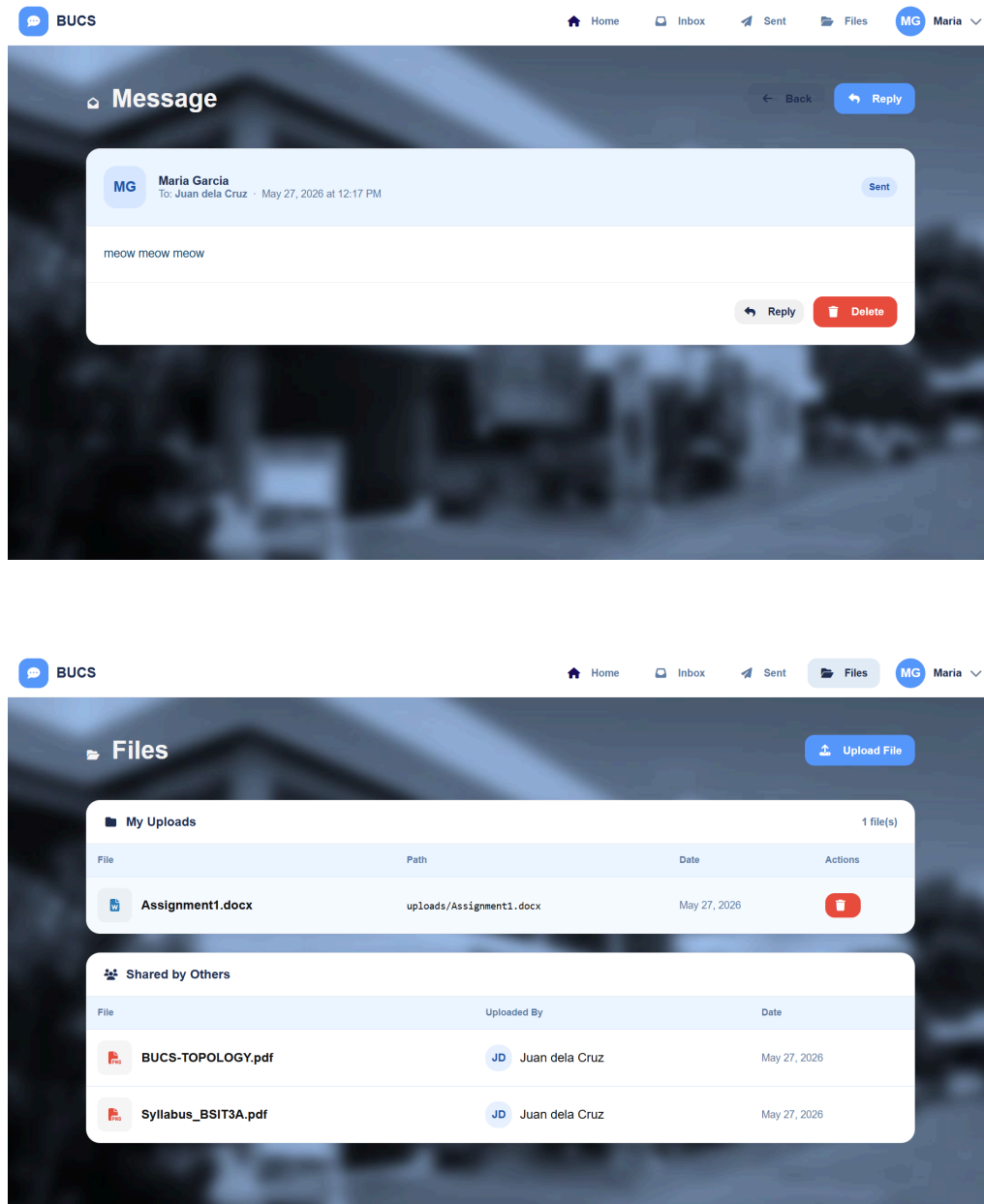


Figure 5. Delete Function Interface

5. Individual Reflection

Questions	Members
1. What was the most difficult part of interpreting the original author's logic? 2. What technical challenges did you face during recreation? 3. How would you improve this module further?	In my situation, I had a difficult time interpreting the author's design because I didn't get to see their original system design; as a result, we had to think of and create our own from scratch. However, I think it was ultimately beneficial that we had the freedom to design the system. Even though it was more challenging, it allowed us to test our capabilities as developers. There were also several technical challenges that I faced during the recreation process. First, I had difficulty figuring out how to use PHP because we chose it as our backend language since it was familiar to us, but it was still difficult to understand within such a short period of time. We had previously created a system with CRUD functionality using Java, which was much easier for us to use. However, the author also used Java to develop the original system, which made the recreation more difficult for us because we had to build the system from scratch. Second, I encountered a problem with XAMPP, the server software used to run our system. It was my first time encountering such an error, but fortunately, there was a lot of information available online that helped me solve it. This module can still be improved by adding features such as voice messaging and calling to make communication easier

	and more efficient. Security is also a concern, so future developers should focus on making the system safer and more secure. I also recommend implementing real-time notifications for a better user experience (UX). - <i>Marley D. Besimo</i> Deciphering the original author's logic was tough because it was hard to visualize the UI/UX. The biggest struggle was trying to merge all the different parts of the project without making the code and the interface a "rambled" mess. It felt like putting together a puzzle where the pieces didn't always want to fit, requiring us to think carefully about how to connect the backend work to what the user actually sees on the screen. On the technical side, our biggest headache was the XAMPP server constantly disconnecting. This made the recreation process very frustrating because the tools we relied on kept shutting down in the middle of our work. Every time the connection dropped, it broke our focus and forced us to restart the services, which slowed down our progress and made it difficult to test the features we were building. To improve this module further, we want to focus on making the UI/UX much cleaner and more modern. We also plan to polish the CRUD features to make sure that adding, viewing, updating, and deleting
--	--

	information works perfectly every time. By making the design easier to use and the data management more reliable, we can turn this project into a much stronger and more professional application. <i>- Natalie A. Gebilaguin</i> The most difficult part of interpreting the original author's logic was understanding the message flow between the interface and the database since the system developed in 2010 used older development patterns. We had to carefully trace how messages are created, stored, retrieved, updated, and deleted to properly recreate the functionality. During development, I faced challenges in contributing to the technical tasks due to other academic workloads and limited background in PHP and CSS. I coordinated with my group members to better understand the system and identify my role. Despite this, we maintained good communication and divided tasks effectively into back-end, front-end, and documentation, which allowed us to complete the project successfully. This module can still be improved by adding features such as voice messaging, calling, video chat, and real-time notifications to enhance communication and UX. Security should also be strengthened through proper authentication and data protection. Additionally, the UI/UX can be made more modern, accessible including for users with disabilities, and the CRUD
--	---

	operations further optimized for more reliable data management. - <i>Mishli Valorrhi N. Postrado</i>
	The most challenging part of understanding the original system's logic was the messaging feature that allowed users to send and receive messages from one another. Since I only had basic knowledge of PHP and limited experience making it responsive with other languages, this aspect became the most difficult part of building the system. Similarly, I encountered technical challenges when connecting the design to the database. There were several processes I was not yet familiar with, and I struggled to fully understand them at first. However, working with my groupmates made the task more manageable. Through constant communication and mutual support, we helped each other grasp the different components of the system, which lightened both the workload and the pressure. We plan to improve this module by adding more interactive features for student communication. One improvement would be creating a group chat for each class where announcements and messages can be shared in one place. I would also develop a feed where students can start topics, share thoughts, and upload images, videos, files, and music related to academic discussions. To further support different communication preferences, I would include options for calls, video chats, and voice recordings.

	These additions would make the platform more dynamic and better suited for collaborative learning. - <i>Shieanne R. Sulpico</i>
--	---